

TP – Films (Partie 1)

Dans ce TP nous allons commencer à étudier les ressources qui permettent de créer des routes « CRUD » (**C**reate, **R**ead, **U**ppdate, **D**eleate) adaptées à la persistance de données. Comme exemple pratique nous allons prendre le cas d'une table de films.

Les données

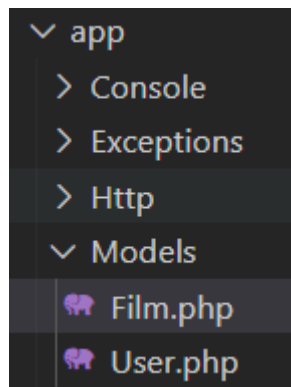
On repart d'un Laravel vierge et on crée une base comme on l'a vu précédemment. On renseigne le fichier **.env** en conséquence :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=tp_film
DB_USERNAME=root
DB_PASSWORD=
```

La migration

On va créer avec Artisan le modèle **Film** en même temps que la migration :

```
php artisan make:model Film --migration
```



Pour faire simple on va se contenter de 3 colonnes pour le titre du film, son année de sortie et sa description :

```
public function up()
{
    Schema::create('films', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->year('year');
```

```
        $table->text('description');
        $table->timestamps();
    });
}
```

On a les champs :

- **id** : entier auto-incrémenté qui sera la clé primaire de la table,
- **title** : texte pour le nom du film,
- **year** : année de sortie du film,
- **description** : description du film,
- **created_at** et **updated_at** créés par la méthode **timestamps**,

Ensuite on lance la migration :

```
php artisan migrate
```

On a la création des tables de base de Laravel qu'on a déjà vues et qui ne nous intéressent pas pour cet article. Mais on voit aussi la création de la table **films** :

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
1	id 	bigint(20)		UNSIGNED	Non	Aucun(e)		AUTO_INCREMENT
2	title	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)		
3	year	year(4)			Non	Aucun(e)		
4	description	text	utf8mb4_unicode_ci		Non	Aucun(e)		
5	created_at	timestamp			Oui	NULL		
6	updated_at	timestamp			Oui	NULL		

Le modèle

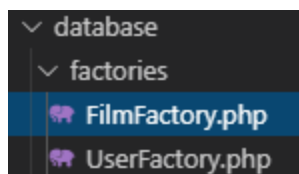
Le modèle **Film** qu'on a créé est vide au départ (il n'y a que le trait pour le factory). On va se contenter de prévoir l'assignement de masse avec la propriété **\$fillable** :

```
protected $fillable = ['title', 'year', 'description'];
```

La population

Pour nos essais on va remplir un peu la table avec quelques films. On va créer un factory :

```
php artisan make:factory FilmFactory
```



On va changer le code de la fonction *definition* :

```
public function definition()
{
    return [
        'title' => $this->faker->sentence(2, true),
        'year' => $this->faker->year,
        'description' => $this->faker->paragraph(),
    ];
}
```

On change ensuite le code du seeder (**database/seeds/DatabaseSeeder.php**) :

```
use App\Models\Film;
...
public function run()
{
    // \App\Models\User::factory(10)->create();
    Film::factory(10)->create();
}
```

Il ne reste plus qu'à lancer la population :

```
php artisan db:seed
```

Si tout va bien on se retrouve avec 10 films dans la table :

id	title	year	description
1	Consequatur quisquam voluptatibus.	2008	Eum aperiam amet ipsam dolorem non. Libero nemo qu...
2	Impedit pariatur.	2007	Veniam consequuntur beatae expedita reprehenderit ...
3	Debitis aut repellat.	1977	Ipsam fugit id et veniam aut facere. Enim doloremq...
4	Vitae quas sequi.	1982	Perferendis consectetur et ullam et. Dolor est nos...
5	Tempora voluptas.	1986	Iure ullam sapiente sit voluptas ea amet non hic. ...
6	Dolores occaecati.	1985	Sed aut nulla autem fugiat aut delectus temporibus...
7	Rerum rerum rerum.	2000	Non earum veniam officiis id dolore officiis. Volu...
8	Amet rerum illum.	2010	Eum distinctio dignissimos harum ut eos. Quibusdam...
9	Occaecati et.	2015	Voluptatem quo iusto sunt. Alias corrupti laborum ...
10	Qui qui labore.	1973	Distinctio est et incidunt distinctio nobis. Eum e...

Une ressource

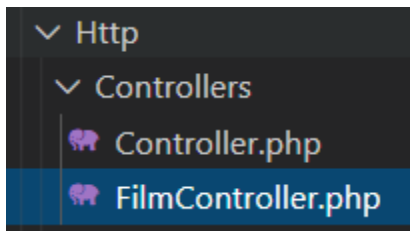
Le contrôleur

On va maintenant créer un contrôleur de ressource avec Artisan :

```
php artisan make:controller FilmController --resource
```

C'est la commande qu'on a déjà vue pour créer un contrôleur avec en plus l'option **--resource**.

Vous trouvez comme résultat le contrôleur **app/Http/Controllers/FilmController** :



Les 7 méthodes créées couvrent la gestion complète des films:

- **index** : pour afficher la liste des films,
- **create** : pour envoyer le formulaire pour la création d'un nouveau film,
- **store** : pour créer un nouveau film,
- **show** : pour afficher les données d'un film,
- **edit** : pour envoyer le formulaire pour la modification d'un film,
- **update** : pour modifier les données d'un film,
- **destroy** : pour supprimer un film.

Les routes

Pour créer toutes les routes il suffit de cette unique ligne de code dans le fichier **web.php** :

```
Route::resource('films', FilmController::class);
```

On va vérifier ces routes avec Artisan :

```
php artisan route:list -v
```

```
GET|HEAD      films ..... films.index > FilmController@index
POST          films ..... films.store > FilmController@store
GET|HEAD      films/create ..... films.create > FilmController@create
GET|HEAD      films/{film} ..... films.show > FilmController@show
PUT|PATCH    films/{film} ..... films.update > FilmController@update
DELETE        films/{film} ..... films.destroy > FilmController@destroy
GET|HEAD      films/{film}/edit ..... films.edit > FilmController@edit
GET|HEAD      sanctum/csrf-cookie ..... Laravel\Sanctum > CsrfCookieController@show
```

Vous trouvez 7 routes, avec chacune une méthode et une url, qui pointent sur les 7 méthodes du contrôleur. Notez également que chaque route a aussi un nom qui peut être utilisé par exemple pour une redirection.

On peut obtenir plus d'information avec la commande « verbose » :

```
php artisan route:list -v
```

```
GET|HEAD      films ..... films.index > FilmController@index
               [] web
POST          films ..... films.store > FilmController@store
               [] web
GET|HEAD      films/create ..... films.create > FilmController@create
               [] web
GET|HEAD      films/{film} ..... films.show > FilmController@show
               [] web
PUT|PATCH    films/{film} ..... films.update > FilmController@update
               [] web
DELETE        films/{film} ..... films.destroy > FilmController@destroy
               [] web
GET|HEAD      films/{film}/edit ..... films.edit > FilmController@edit
               [] web
GET|HEAD      sanctum/csrf-cookie ..... Laravel\Sanctum > CsrfCookieController@show
               [] web
```

On retrouve aussi pour chaque route le middleware **web** dont je vous ai déjà parlé.

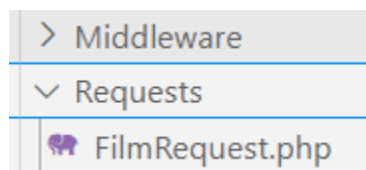
*Le middleware **web** est automatiquement ajouté à toutes les routes du fichier **routes/web.php**.*

Nous allons à présent considérer chacune de ces routes et créer la gestion des données, les vues, et le code nécessaire au niveau du contrôleur.

La validation

On va créer une requête de formulaire pour la création ou la modification d'un film :

```
php artisan make:request FilmRequest
```



On a deux champs à vérifier : le titre et l'année. A priori il n'y a aucune différence de validation pour la création et la modification. Voilà le code modifié pour cette classe :

```
class FilmRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     *
     * @return bool
     */
    public function authorize()
    {
        return false;
    }
}
```

```

    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, mixed>
     */
    public function rules()
    {
        return [
            'title' => ['required', 'string', 'max:100'],
            'year' => ['required', 'numeric', 'min:1950', 'max:' . date('Y')],
            'description' => ['required', 'string', 'max:500'],
        ];
    }
}

```

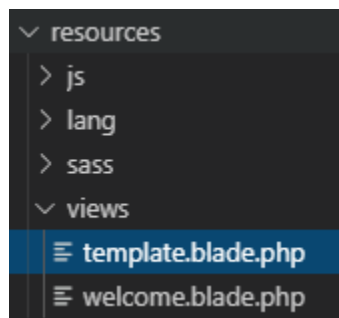
On prévoit comme règles :

- **title** : le champ est obligatoire (**required**), ça doit être du texte (**string**), le nombre maximum de caractères (**max**) doit être de 100
- **year** : le champ est obligatoire (**required**), ça doit être un nombre (**number**), la valeur minimale (**min**) doit être 1950, la valeur maximale (**max**) doit être l'année actuelle (date('Y'))
- **description** : le champ est obligatoire (**required**), ça doit être du texte (**string**), le nombre maximum de caractères (**max**) doit être de 500

On pourra injecter cette classe dans les méthodes du contrôleur.

Le template

Laravel s'occupe essentiellement du côté serveur et n'impose rien côté client, même s'il propose des choses. Autrement dit on peut utiliser Laravel avec n'importe quel système côté client. Pour notre exemple je vous propose d'utiliser **Bulma** pour la mise en forme. Voici un template qui va nous servir pour toutes nos vues :



```

<!DOCTYPE html>
<html>

<head>

```

```

<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Films</title>
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bulma@0.9.3/css/bulma.min.css">
  @yield('css')
</head>

<body>
  <main class="section">
    <div class="container">
      @yield('content')
    </div>
  </main>
</body>

</html>

```

La liste des films

La route

La liste des films correspond à cette route :

```
GET|HEAD      films ..... films.index > FilmController@index
```

Le contrôleur

Dans le contrôleur c'est la méthode **index** qui est concernée. On va donc la coder :

```

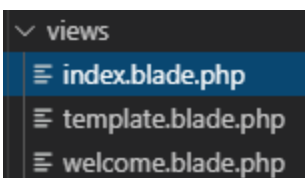
public function index()
{
    $films = Film::paginate(5);
    return view('index', compact('films'));
}

```

On va chercher tous les films avec la méthode **all** du modèle, on appelle la vue **index** en lui transmettant les films.

La vue index

On crée la vue **index** :



Avec ce code :

```
@extends('template')
@section('content')
    <div class="card">
        <header class="card-header">
            <p class="card-header-title">Films</p>
        </header>
        <div class="card-content">
            <div class="content">
                <table class="table is-hoverable">
                    <thead>
                        <tr>
                            <th>#</th>
                            <th>Titre</th>
                            <th></th>
                            <th></th>
                            <th></th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach ($films as $film)
                            <tr>
                                <td>{{ $film->id }}</td>
                                <td><strong>{{ $film->title }}</strong></td>
                                <td><a class="button is-primary" href="{{
route('films.show', $film->id) }}">Voir</a></td>
                                <td><a class="button is-warning" href="{{
route('films.edit', $film->id) }}">Modifier</a></td>
                                <td>
                                    <form action="{{ route('films.destroy', $film->id) }}"
method="post">
                                        @csrf
                                        @method('DELETE')
                                        <button class="button is-danger"
type="submit">Supprimer</button>
                                    </form>
                                </td>
                            </tr>
                        @endforeach
                    </tbody>
                </table>
            </div>
        </div>
        <footer class="card-footer">
            {{ $films->links() }}
        </footer>
    </div>
@endsection
```


Films				
#	Titre			
1	Aspernatur quo illo.	Voir	Modifier	Supprimer
2	Officia illum eos.	Voir	Modifier	Supprimer
3	Doloremque architecto.	Voir	Modifier	Supprimer
4	Ea est.	Voir	Modifier	Supprimer
5	Consequuntur voluptatem modi.	Voir	Modifier	Supprimer
6	Voluptates enim.	Voir	Modifier	Supprimer
7	Porro in.	Voir	Modifier	Supprimer
8	Tempora non.	Voir	Modifier	Supprimer
9	Repellat et.	Voir	Modifier	Supprimer
10	Provident qui.	Voir	Modifier	Supprimer

Quelques remarques concernant le code :

- on utilise la directive **@foreach** pour faire une boucle sur tous les films
- la méthode **route** qui génère une url selon la route peut être accompagnée d'un paramètre, par exemple **route('films.show', \$film->id)** permet de générer l'url de la forme **.../films/id**
- les formulaire HTML ne supportent pas les verbes PUT, PATCH et DELETE, du coup on doit utiliser le verbe POST et prévoir dans le formulaire un input caché qui indique le verbe à utiliser en réalité, la directive **@method** permet de facilement mettre ça en place, on s'en sert pour le bouton de suppression

La pagination

Ici on n'a que 10 films mais imaginez qu'on en ait des centaines ou des milliers ! Dans ce cas une pagination serait la bienvenue. Laravel est bien équipé pour ça.

On a remplacé la méthode **all** par **paginate** en indiquant en paramètre le nombre d'enregistrement par page. Ensuite dans la vue il suffit de prévoir ce code :

```
</div>
<footer class="card-footer">
{{ $films->links() }}
</footer>
</div>
@endsection
```

Mais le souci c'est que par défaut le marquage généré est celui qui convient à Tailwind, du coup avec Bulma on obtient ça :

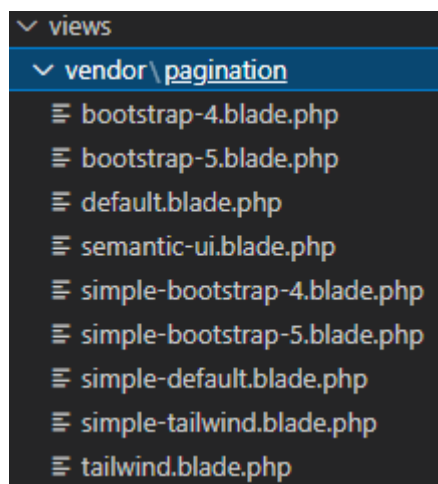
```
« Previous Next »
Showing 1 to 5 of 10 results
```

Ce qui n'est pas du meilleur effet !

Il nous faut donc modifier la vue qui génère ce code. Comme nous ne sommes pas les premiers à avoir ce souci il suffit de chercher sur la toile et on trouve facilement [ce repo](#).

On commence par publier les vues :

```
php artisan vendor:publish --tag=laravel-pagination
```



On se rend compte d'ailleurs qu'il y en a une de prévue pour **Semantic** qui est aussi un superbe framework, de même pour **Bootstrap** qui n'est plus à présenter.

Pour terminer on remplace le code du fichier **tailwind.blade.php** par celui-ci :

```

@if ($paginator->hasPages())
    <nav class="pagination is-centered" role="navigation" aria-label="pagination">
        {{-- Previous Page Link --}}
        @if ($paginator->onFirstPage())
            <a class="pagination-previous" disabled>@lang('pagination.previous')</a>
        @else
            <a class="pagination-previous" href="{{ $paginator->previousPageUrl()
        }}">@lang('pagination.previous')</a>
        @endif
        {{-- Next Page Link --}}
        @if ($paginator->hasMorePages())
            <a class="pagination-next" href="{{ $paginator->nextPageUrl()
        }}">@lang('pagination.next')</a>
        @else
            <a class="pagination-next" disabled>@lang('pagination.next')</a>
        @endif
        {{-- Pagination Elements --}}
        <ul class="pagination-list">
            @foreach ($elements as $element)
                {{-- "Three Dots" Separator --}}
                @if (is_string($element))
                    <li><span class="pagination-ellipsis">&hellip;</span></li>
                @endif
                {{-- Array Of Links --}}
                @if (is_array($element))
                    @foreach ($element as $page => $url)
                        @if ($page == $paginator->currentPage())
                            <li><a class="pagination-link is-current"
                                aria-label="Goto page {{ $page }}">{{ $page }}</a></li>
                        @else
                            <li><a href="{{ $url }}" class="pagination-link"
                                aria-label="Goto page {{ $page }}">{{ $page }}</a></li>
                        @endif
                    @endforeach
                @endif
            @endforeach
        </ul>
    </nav>
@endif

```

Maintenant c'est un peu mieux :

5	Consequuntur voluptatem modi.	Voir	Modifier	Supprimer
Previous	1	2	Next Page	

Ça mérite juste d'être un peu aéré et centré. Comme Bulma utilise Flex on va ajouter quelques règles dans la vue **index** :

```
@section('css')
    <style>
        .card-footer {
            justify-content: center;
            align-items: center;
            padding: 0.4em;
        }
    </style>
@endsection
```

Maintenant c'est plus joli :



L'affichage d'un film

On va voir maintenant l'affichage les données d'un film. On y accède à partir du bouton **Voir**.

La route

La liste des films correspond à cette route :

```
GET|HEAD      films/{film} ..... films.show > FilmController@show
```

Le contrôleur

Dans le contrôleur c'est la méthode **show** qui est concernée. On va donc la coder.

Il me faut toutefois préciser déjà un point important. Dans la version du contrôleur générée par défaut on voit que le film au niveau des arguments des fonctions est référencé par son identifiant, par exemple :

```
public function show($id)
```

La variable **id** contient la valeur passée dans l'url. Par exemple **.../films/8** indique qu'on veut voir les informations du film d'identifiant 8. Il suffit donc ensuite d'aller chercher dans la base le film correspondant.

On va utiliser une autre stratégie :

```
public function show(Film $film)
```

L'argument cette fois est une instance du modèle **App\Models\Film**. Etant donné qu'il rencontre ce type, Laravel va automatiquement livrer une instance du modèle pour le film concerné ! C'est ce

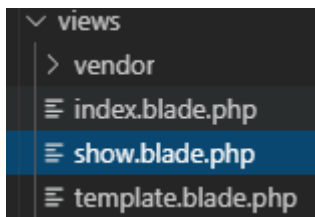
qu'on appelle liaison implicite (Implicit Binding). Vous voyez encore là à quel point Laravel nous simplifie la vie.

Du coup la méthode devient très simple à coder :

```
public function show(Film $film)
{
    return view('show', compact('film'));
}
```

La vue show

On crée cette vue :



Avec ce code :

```
@extends('template')
@section('content')
    <div class="card">
        <header class="card-header">
            <p class="card-header-title">Titre : {{ $film->title }}</p>
        </header>
        <div class="card-content">
            <div class="content">
                <p>Année de sortie : {{ $film->year }}</p>
                <hr>
                <p>{{ $film->description }}</p>
            </div>
        </div>
    </div>
@endsection
```

Titre : Aspernatur quo illo.

Année de sortie : 1981

Ut tempora nobis et dolorem. Ut et commodi laudantium atque reprehenderit aut ab. Et perferendis ipsum enim non modi aperiam consectetur.

Sobre et efficace. J'aurais pu prévoir un bouton de retour mais les navigateurs font déjà ça très bien.

Supprimer un film

La route

La suppression d'un film correspond à cette route :

```
DELETE      films/{film} ..... films.destroy > FilmController@destroy
```

Le contrôleur

Dans le contrôleur c'est la méthode **destroy** qui est concernée. On va donc la coder :

```
public function destroy(Film $film)
{
    $film->delete();
    return back()->with('info', 'Le film a bien été supprimé dans la base de
données.');
```

Comme pour la méthode show on utilise une liaison implicite et on obtient du coup immédiatement une instance du modèle. Comme c'est un peu brutal comme suppression il peut être judicieux de fournir un message de confirmation pour l'utilisateur. Je ne vais pas le faire ici pour ne pas trop alourdir le projet et il s'agit uniquement de traitement côté client.

Par contre après la suppression il faut afficher quelque chose pour dire que l'opération s'est réalisée correctement. On voit qu'il y a une redirection avec la méthode **back** qui renvoie la même page. D'autre part la méthode **with** permet de flasher une information dans la session. Cette information ne sera valide que pour la requête suivante. Dans notre vue **index** on va prévoir quelque chose pour afficher cette information :

```
@section('content')
    @if (session()->has('info'))
        <div class="notification is-success">
            {{ session('info') }}
        </div>
    @endif
```

La directive **@if** permet de déterminer si une information est présente en session, et si c'est le cas de l'afficher :

Le film a bien été supprimé dans la base de données.

Films

Classiquement on prévoit un bouton pour fermer la barre de notification. Là encore je vais simplifier parce c'est aussi un traitement purement client. Il suffit d'ajouter un peu de Javascript pour le faire (vous pouvez consulter [la documentation de Bulma sur le sujet](#)).

Dans le prochain article on verra comment créer et modifier un film.

Créer un film

Les routes

Pour la création d'un film on va avoir deux routes :

- Pour afficher le formulaire de création
- Pour soumettre le formulaire

```
POST      films ..... films.store › FilmController@store
GET|HEAD  films/create ..... films.create › FilmController@create
```

Le contrôleur

Dans le contrôleur ce sont les méthodes **create** et **store** qui sont concernées. On va donc les coder :

```
public function create()
{
    return view('create');
}

public function store(FilmRequest $filmRequest)
{
    Film::create($filmRequest->all());
    return redirect()->route('films.index')->with('info', 'Le film a bien été créé');
}
```

La vue index

On a déjà codé la vue **index** mais on n'a pas prévu de bouton pour créer un film. Alors on va arranger ça (je remets tout le code pour que ce soit plus simple) :

```
@extends('template')
@section('css')
<style>
    .card-footer {
        justify-content: center;
        align-items: center;
        padding: 0.4em;
    }
</style>
</section>
```

```

        .is-info {
            margin: 0.3em;
        }
    </style>
@endsection
@section('content')
    @if (session()->has('info'))
        <div class="notification is-success">
            {{ session('info') }}
        </div>
    @endif
    <div class="card">
        <header class="card-header">
            <p class="card-header-title">Films</p>
            <a class="button is-info" href="{{ route('films.create') }}">Créer un film</a>
        </header>
        <div class="card-content">
            <div class="content">
                <table class="table is-hoverable">
                    <thead>
                        <tr>
                            <th>#</th>
                            <th>Titre</th>
                            <th></th>
                            <th></th>
                            <th></th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach ($films as $film)
                            <tr>
                                <td>{{ $film->id }}</td>
                                <td><strong>{{ $film->title }}</strong></td>
                                <td><a class="button is-primary" href="{{
route('films.show', $film->id) }}">Voir</a></td>
                                <td><a class="button is-warning" href="{{
route('films.edit', $film->id) }}">Modifier</a>
                                </td>
                                <td>
                                    <form action="{{ route('films.destroy', $film->id) }}"
method="post">
                                        @csrf
                                        @method('DELETE')
                                        <button class="button is-danger"
type="submit">Supprimer</button>
                                    </form>
                                </td>
                            </tr>
                        @endforeach
                    </tbody>
                </table>
            </div>
        </div>
    </div>

```

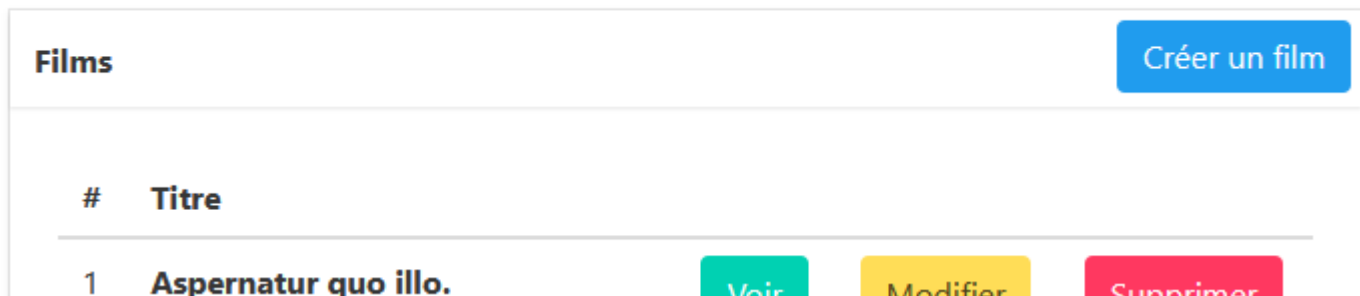


```

        </table>
    </div>
</div>
<footer class="card-footer is-centered">
    {{ $films->links() }}
</footer>
</div>
@endsection

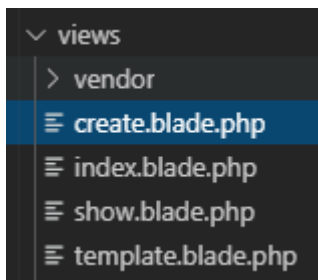
```

Maintenant on a un bouton :



La vue create

On crée la vue **create** :



```

@extends('template')
@section('content')
    <div class="card">
        <header class="card-header">
            <p class="card-header-title">Création d'un film</p>
        </header>
        <div class="card-content">
            <div class="content">
                <form action="{{ route('films.store') }}" method="POST">
                    @csrf
                    <div class="field">
                        <label class="label">Titre</label>
                        <div class="control">
                            <input class="input @error('title') is-danger @enderror"
type="text" name="title"
                                value="{{ old('title') }}" placeholder="Titre du film">
                        </div>
                    </div>
                </form>
            </div>
        </div>
    </div>

```

```

        @error('title')
            <p class="help is-danger">{{ $message }}</p>
        @enderror
    </div>
    <div class="field">
        <label class="label">Année de diffusion</label>
        <div class="control">
            <input class="input" type="number" name="year" value="{{
old('year') }}" min="1950"
                max="{{ date('Y') }}">
        </div>
        @error('year')
            <p class="help is-danger">{{ $message }}</p>
        @enderror
    </div>
    <div class="field">
        <label class="label">Description</label>
        <div class="control">
            <textarea class="textarea" name="description"
placeholder="Description du film">{{ old('description') }}</textarea>
        </div>
        @error('description')
            <p class="help is-danger">{{ $message }}</p>
        @enderror
    </div>
    <div class="field">
        <div class="control">
            <button class="button is-link">Envoyer</button>
        </div>
    </div>
</form>
</div>
</div>
</div>
@endsection

```

Création d'un film

Titre

Année de diffusion

Description

Description du film

Envoyer

Lorsqu'un film a été créé on retourne à la vue **index** avec un message :

Le film a bien été créé

Films		Créer un film		
#	Titre			
1	Aspernatur quo	Voir	Modifier	Supprimer

Modifier un film

Les routes

Pour la modification d'un film on va avoir deux routes :

- Pour afficher le formulaire de modification
- Pour soumettre le formulaire

```
PUT|PATCH  films/{film} ..... films.update > FilmController@update
DELETE      films/{film} ..... films.destroy > FilmController@destroy
GET|HEAD    films/{film}/edit ..... films.edit > FilmController@edit
```

Le contrôleur

Dans le contrôleur ce sont les méthodes **edit** et **update** qui sont concernées. On va donc les coder :

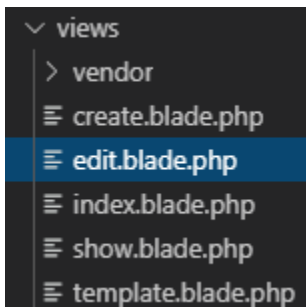
```
public function edit(Film $film)
{
    return view('edit', compact('film'));
}

public function update(FilmRequest $filmRequest, Film $film)
{
    $film->update($filmRequest->all());
    return redirect()->route('films.index')->with('info', 'Le film a bien été
modifié');
}
```

Le code ressemble beaucoup à celui vu ci-dessus pour la création et c'est normal.

La vue edit

Là aussi on va avoir pratiquement le même code que la vue **create**. La différence c'est qu'il faut renseigner les contrôles du formulaire au départ. On crée la vue **edit** :



```

@extends('template')
@section('content')
    <div class="card">
        <header class="card-header">
            <p class="card-header-title">Modification d'un film</p>
        </header>
        <div class="card-content">
            <div class="content">
                <form action="{{ route('films.update', $film->id) }}" method="POST">
                    @csrf
                    @method('put')
                    <div class="field">
                        <label class="label">Titre</label>
                        <div class="control">
                            <input class="input @error('title') is-danger @enderror"
type="text" name="title"
                                value="{{ old('title', $film->title) }}" placeholder="Titre
du film">
                        </div>
                        @error('title')
                            <p class="help is-danger">{{ $message }}</p>
                        @enderror
                    </div>
                    <div class="field">
                        <label class="label">Année de diffusion</label>
                        <div class="control">
                            <input class="input" type="number" name="year" value="{{
old('year', $film->year) }}"
                                min="1950" max="{{ date('Y') }}">
                        </div>
                        @error('year')
                            <p class="help is-danger">{{ $message }}</p>
                        @enderror
                    </div>
                    <div class="field">
                        <label class="label">Description</label>
                        <div class="control">
                            <textarea class="textarea" name="description"
placeholder="Description du film">{{ old('description', $film->description) }}</textarea>
                        </div>
                        @error('description')
                            <p class="help is-danger">{{ $message }}</p>
                        @enderror
                    </div>
                    <div class="field">
                        <div class="control">
                            <button class="button is-link">Envoyer</button>
                        </div>
                    </div>
                </form>
            </div>
        </div>
    </div>

```

```
</div>  
</div>  
@endsection
```

Modification d'un film

Titre

Année de diffusion

Description

Ut tempora nobis et dolorem. Ut et commodi laudantium atque reprehenderit aut ab.
Et perferendis ipsum enim non modi aperiam consectetur.

Là encore on a la validation qui fonctionne et le message qui informe de la modification avec la redirection :

Le film a bien été modifié

Ordonner les films

Pour le moment les films sont listés dans l'ordre de leur identifiant, ce qui n'est pas très heureux. Il serait plus judicieux d'avoir la liste dans l'ordre alphabétique des noms de films. Pour le faire il suffit d'intervenir dans le contrôleur :

```
public function index()
{
    $films = Film::oldest('title')->paginate(5);
    return view('index', compact('films'));
}
```

Maintenant on a bien les films dans l'ordre désiré :

Films	
#	Titre
8	Accusantium fugit sed.
1	Dolor animi at.
7	Dolor necessitatibus ut.
6	Eum eligendi.
9	Molestias ipsum error.

Finalement aussi quel intérêt de faire apparaître l'identifiant ? Surtout maintenant qu'ils sont dans le désordre, on va donc modifier la vue **index** :

Films				Créer un film
Titre				
Accusantium fugit sed.	Voir	Modifier	Supprimer	
Dolor animi at.	Voir	Modifier	Supprimer	
Dolor necessitatibus ut.	Voir	Modifier	Supprimer	
Eum eligendi.	Voir	Modifier	Supprimer	
Molestias ipsum error.	Voir	Modifier	Supprimer	
« Précédent 1 2 Suivant »				

Soft delete

Telle qu'on a prévu la suppression des films ils sont définitivement retirés de la base. Laravel propose une autre façon de procéder avec le **soft delete**. Dans un premier temps on se contente de marquer le film comme supprimé en renseignant une colonne **deleted_at**. Dans un deuxième temps on peut faire la suppression définitive. En gros on crée une corbeille.

La migration

Dans la migration de la table films on ajoute la colonne avec la méthode **softDeletes** :

```
public function up()
{
    Schema::create('films', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->year('year');
        $table->text('description');
        $table->timestamps();
        $table->softDeletes();
    });
}
```

On relance la migration avec la population :

```
php artisan migrate:fresh --seed
```


Le modèle

Au niveau du modèle Film on ajoute le trait **SoftDeletes** :

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Film extends Model
{
    use HasFactory, SoftDeletes;
    protected $fillable = ['title', 'year', 'description'];
}
```

Maintenant quand on supprime un film on le conserve dans la base et on renseigne la colonne **deleted_at** :

3	Voluptatibus dicta.	1970	NULL
4	Consequatur vitae consectetur.	2012	2022-02-03 16:18:59

Les routes

On ajoute deux routes pour les deux nouvelles actions :

```
Route::controller(FilmController::class)->group(function () {
    Route::delete('films/force/{film}', 'forceDestroy')->name('films.force.destroy');
    Route::put('films/restore/{film}', 'restore')->name('films.restore');
});
```

DELETE	films/force/{film}		films.force.destroy	»	FilmController@forceDestroy
PUT	films/restore/{film}		films.restore	»	FilmController@restore

Le contrôleur

Dans le contrôleur il faut déjà modifier la méthode **index** pour inclure les films de la corbeille :

```
public function index()
{
    $films = Film::withTrashed()->oldest('title')->paginate(5);
    return view('index', compact('films'));
}
```

Dans la méthode **destroy** il faut changer le texte du message :

```
return back()->with('info', 'Le film a bien été mis dans la corbeille.');
```

Enfin il faut créer les deux nouvelles méthodes :

```
public function forceDestroy($id)
{
    Film::withTrashed()->whereId($id)->firstOrFail()->forceDelete();
    return back()->with('info', 'Le film a bien été supprimé définitivement dans la
base de données.');
```

```
public function restore($id)
{
    Film::withTrashed()->whereId($id)->firstOrFail()->restore();
    return back()->with('info', 'Le film a bien été restauré.');
```

La vue index

Il ne reste plus qu'à modifier la vue **index** :

```
@foreach ($films as $film)
    <tr @if ($film->deleted_at) class="has-background-grey-lighter" @endif>
        <td><strong>{{ $film->title }}</strong></td>
        <td>
            @if ($film->deleted_at)
                <form action="{{ route('films.restore', $film->id) }}" method="post">
                    @csrf
                    @method('PUT')
                    <button class="button is-primary" type="submit">Restaurer</button>
                </form>
            @else
                <a class="button is-primary" href="{{ route('films.show', $film->id)
}}} ">Voir</a>
            @endif
        </td>
        <td>
            @if ($film->deleted_at)
            @else
                <a class="button is-warning" href="{{ route('films.edit', $film->id)
}}} ">Modifier</a>
            @endif
        </td>
        <td>
            <form action="{{ route($film->deleted_at ? 'films.force.destroy' :
'films.destroy', $film->id) }}" method="post">
```

```

        @csrf
        @method('DELETE')
        <button class="button is-danger" type="submit">Supprimer</button>
    </form>
</td>
</tr>
@endforeach

```

Fonctionnement

Maintenant quand on clique sur le bouton **Supprimer** d'un film ça le met dans la corbeille et il prend cette apparence avec un fond grisé et des boutons différents :

Titre			
Consequatur quas veritatis.	Voir	Modifier	Supprimer
Consequuntur dolores adipisci.	Restaurer		Supprimer
Dicta commodi.	Voir	Modifier	Supprimer

On peut alors soit le supprimer définitivement avec le bouton **Supprimer**, soit le restaurer avec le bouton **Restaurer**.